

ObRail Europe — Intégration Machine Learning, stabilisation et revue de code

Pull Request #4 — branche merge/ml-into-main vers main

15 juin 2026

Table des matières

1. Objectif	2
2. Contenu fonctionnel intégré	2
2.1 Conflit de merge résolu	2
3. Stabilisation (bugs trouvés et corrigés)	2
3.1 Backend — versions incompatibles (erreur 500 généralisée)	2
3.2 Dashboard — cache empoisonné du Simulateur	2
4. Revue de code — 5 correctifs supplémentaires	4
4.1 (#1) Calcul erroné de la racine projet — <i>latent</i> : 500 sur toutes les prédictions	4
4.2 (#2) Ratios CO ₂ par cluster incohérents avec les labels	4
4.3 (#3) Carte et carte de résultat classaient le même trajet différemment	4
4.4 (#4) Encodage recalculé à chaque prédiction	5
4.5 (#5) Branche morte dans <code>predict_full</code>	5
5. Tests	6
6. Vérification en conditions réelles	6
7. Synthèse des livrables	6

1. Objectif

Fusionner la branche `ml` (intégration *Machine Learning* : modèles, API de prédiction, page Simulateur, nouvelle base de données) avec la dernière version de `main`, **sur une branche dédiée** (`merge/ml-into-main`), puis stabiliser l'ensemble, le couvrir de tests et le soumettre via la **Pull Request #4**.

Le tout a été vérifié en local sur la stack Docker complète (`docker compose up -d --build`), base rechargée depuis le nouveau dump.

2. Contenu fonctionnel intégré

La branche `ml` apporte :

- **Dossier `ml/`** : modèles entraînés (`model_regression.joblib` — XGBoost, `model_clustering.joblib` — KMeans, scalers), notebooks (analyse, *preprocessing*, *training*), jeux de données (raw, processed, splits), scripts `src/` et figures.
- **Backend (FastAPI)** : trois nouvelles routes `POST /predict/emissions`, `/predict/cluster`, `/predict/full` (+ schémas Pydantic + tests).
- **Dashboard (Streamlit)** : nouvelle page **Simulateur CO₂** (`_pages/simulateur.py`) + service d'appel API.
- **Base de données** : nouveau dump `talend/dump/dump_mspr2.sql` (11 tables, rechargé automatiquement à l'initialisation de PostgreSQL).

2.1 Conflit de merge résolu

Un seul conflit, sur `dashboard/app.py` (en-tête de navigation) : `main` utilisait 4 colonnes, `ml` en voulait 5 (onglet « Simulateur » ajouté). Le design de `main` a été conservé puis étendu à 5 colonnes (`st.columns([4, 1.15, 1.15, 1.15, 1.15])`).

3. Stabilisation (bugs trouvés et corrigés)

3.1 Backend — versions incompatibles (erreur 500 généralisée)

`backend/requirements.txt` n'épinglait aucune version. Un *rebuild* tirait donc **Starlette 1.3**, incompatible avec `prometheus-fastapi-instrumentator` (`'_IncludedRouter' object has no attribute 'path'`) : **toutes** les routes incluses renvoyaient une erreur 500.

Correctif : épinglage `fastapi==0.115.12` et `prometheus-fastapi-instrumentator==7.1.0` (Starlette 0.46, compatible).

3.2 Dashboard — cache empoisonné du Simulateur

`_load_segments_with_clusters()` et `_load_operateurs()` étaient décorées `@st.cache_data`. Lorsqu'un appel API échouait ponctuellement (cold start, *timeout*, redémarrage), le **résultat vide était mis en cache pour toute la durée du TTL** (jusqu'à 1 h). Conséquence : le message « Segments non disponibles » restait figé même après le rétablissement de l'API — c'est la cause du symptôme « 1 fois sur 3 » observé en production.

Correctif : ces fonctions **lèvent désormais une exception** sur résultat vide. `st.cache_data` ne mémorise pas les exceptions : l'appel est donc ré-essayé au rendu suivant et se répare tout seul. `render()` rattrape proprement l'exception (message d'attente au lieu d'un *crash*).

4. Revue de code — 5 correctifs supplémentaires

Une revue de code à fort niveau de rappel a été menée sur l'ensemble du diff (~2000 lignes de code). Cinq points ont été relevés puis corrigés.

4.1 (#1) Calcul erroné de la racine projet — *latent* : 500 sur toutes les prédictions

backend/app/routes/predict_routes.py calculait la racine du projet avec un nombre fixe de ... Or la profondeur diffère :

- en local : <repo>/backend/app/routes → racine = <repo>;
- en conteneur : /app/app/routes, ml/ monté sur /app → racine = /app.

Le code résolvait /home/johan (hôte) ou / (conteneur) — un niveau au-dessus de ml/. L'import from ml.api.predict ne réussissait **que par accident**, parce que le répertoire de travail d'uvicorn (/app) était déjà dans sys.path. Lancé depuis un autre répertoire, **chaque** /predict/* **renverrait une 500**.

Correctif : on **remonte l'arborescence** depuis le fichier jusqu'à trouver le dossier contenant ml/api/predict.py. Robuste en local **et** en conteneur, sans dépendre du répertoire de travail.

4.2 (#2) Ratios CO₂ par cluster incohérents avec les labels

_CLUSTER_RATIO_CO2 = {0: 0.03, 1: 0.14, 2: 0.11} produisait une **économie CO₂ affichée** de 97 % / 86 % / 89 % pour les clusters 0 / 1 / 2. Le cluster 2 (« Potentiel limité ») apparaissait donc **plus avantageux** que le cluster 1 (« Potentiel modéré ») — l'inverse de ce que les labels et la légende laissent entendre, et un tri du tableau « Liaisons prioritaires » trompeur.

Correctif : ratios remis dans l'ordre monotone {0: 0.03, 1: 0.11, 2: 0.14} → économie 97 % > 89 % > 86 %, cohérente avec la sévérité des labels. Verrouillé par un test de non-régression.

4.3 (#3) Carte et carte de résultat classaient le même trajet différemment

La carte des liaisons colore les segments par **tranches de distance** (heuristique _estimate_cluster), tandis que la prédiction du formulaire renvoie le **cluster du modèle KMeans**, dont la numérotation n'est pas ordonnée par distance. Exemple : un trajet de 450 km → modèle « Potentiel modéré » (cluster 1), mais la légende « < 600 km » le rangeait en cluster 0. Le partage du libellé « Cluster N » laissait croire à une même identité.

Correctif : la carte n'emploie plus le terme « Cluster N ». Légende, filtre et méta-texte parlent désormais explicitement de **tranches de distance (estimation Haversine)**, présentées comme **distinctes** du cluster KMeans du modèle affiché au-dessus. Plus aucune fausse équivalence.

4.4 (#4) Encodage recalculé à chaque prédiction

`predict_emissions` reconstruisait l'index d'encodage des opérateurs et la moyenne globale à chaque appel.

Correctif : `encoding_map` et `moyenne_globale` sont **pré-calculés une fois** dans `load_models()`. `predict_emissions` les réutilise (avec repli rétro-compatible si absents).

4.5 (#5) Branche morte dans `predict_full`

`ratio = empreinte / empreinte_avion if empreinte_avion > 0 else 0.0:comme distance_km` est validé `gt=0`, `empreinte_avion` est toujours strictement positif. La branche `else` était donc inatteignable.

Correctif : simplification en `ratio = empreinte / empreinte_avion`, avec commentaire rappelant l'invariant de validation.

5. Tests

Couverture ajoutée et statut des suites complètes :

Suite	Tests	Détail
Backend	103	routes ML, contrats, qualité données, stats, trajets
Dashboard	68	dont test_simulateur.py (nouveau) + predict_* dans test_api_service.py
ML	12	clustering + prédiction
Total	183	toutes vertes

Nouveautés notables côté tests :

- dashboard/tests/test_simulateur.py : Haversine, estimation de cluster, cohérence des constantes, génération HTML, **non-régression du fix anti-cache** (un résultat vide doit *lever* et non être mis en cache) et **non-régression du fix #2** (économie monotone décroissante 0 → 1 → 2).
- dashboard/tests/test_api_service.py : couverture des fonctions predict_emissions, predict_cluster, predict_full.

6. Vérification en conditions réelles

Stack relancée via `docker compose up -d --build` (base rechargée depuis le nouveau dump). Contrôles effectués :

- models OK: True — les modèles ML se chargent bien (fix #1 validé en conteneur);
- POST /predict/full (450 km, SNCF, JOUR, 130 min) → **10.08 kg CO₂**, cluster « Potentiel modéré » — HTTP 200;
- carte des liaisons : **319 segments** chargés, plus de « Segments non disponibles ».

7. Synthèse des livrables

- **Branche** merge/ml-into-main, **Pull Request #4** vers main.
- Commits : intégration ML, résolution du merge, fixes de stabilisation + tests, puis les 5 correctifs de revue de code.
- **183 tests** au vert, fixes vérifiés sur la stack Docker.